SkillForge: A Pydantic AI Agent for Autonomous SKILL.md Authoring on mindX

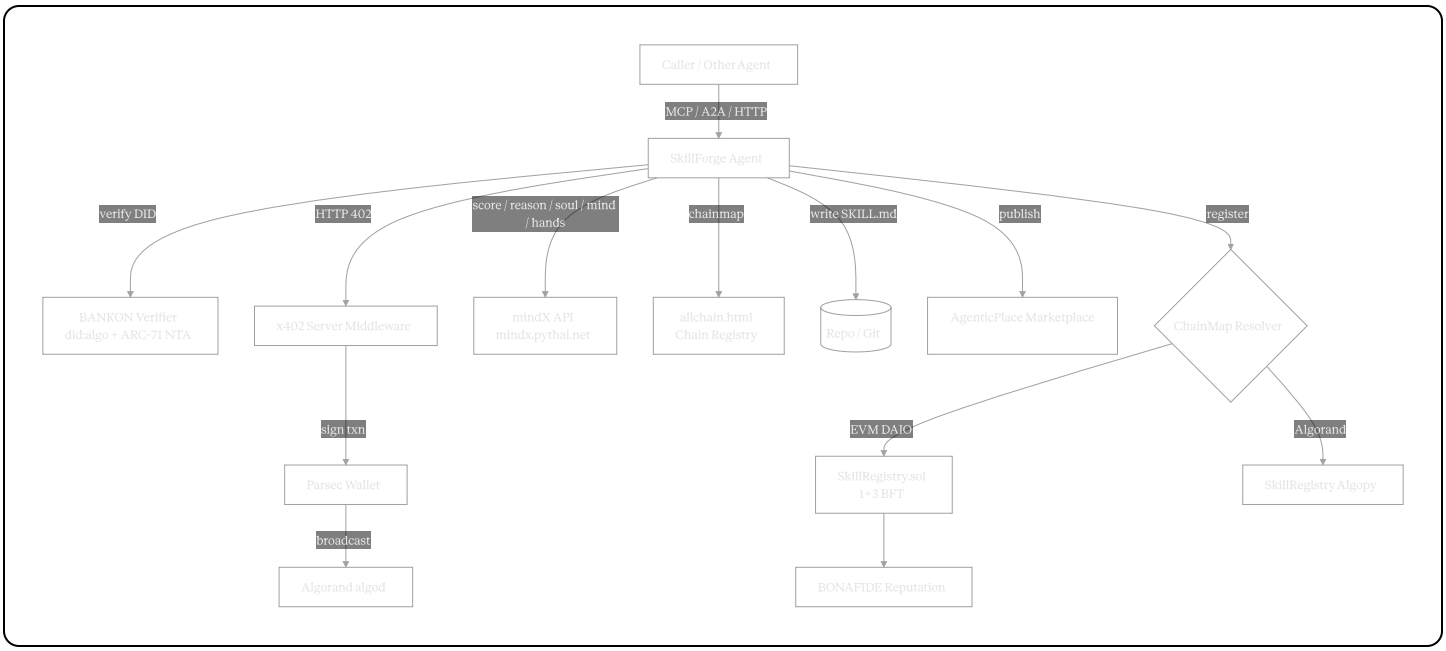
Executive summary

SkillForge is a Pydantic AI agent that authors and improves Matt-Pocock-style SKILL.md files, gated by BANKON identity, paid via x402 on Algorand, scored by mindX, and anchored on-chain through a 2-of-3 BFT-validated SkillRegistry contract. The agent's contract is a typed `SkillMd` Pydantic model — every loop (draft → mindX-score → revise) runs through `@output_validator` + `ModelRetry`, so quality gates *are* the agent loop rather than glue code on top of it. mindX exposes the Soul-Mind-Hands cognitive layers as REST tools; the agent maps **Soul → output validators**, **Mind → reasoning + Pydantic AI**, **Hands → tool execution / file writes / on-chain ops**. The agent is simultaneously consumable as an MCP server (FastMCP, for Claude Code/Cursor), an A2A peer (`agent.to_a2a()`, for AgenticPlace discovery), and a payment-gated HTTP service (x402). Everything that mutates the canonical registry runs through a 1-relayer + 3-validator quorum (Solidity for EVM/DAIO; Algopy for Algorand). The result is a tight loop where the agent *can author skills about itself* — meta-skills modeled directly on Pocock's `improve-codebase-architecture` — and improve them autonomously.

This document delivers: (1) the unified SKILL.md schema as Pydantic v2 with `to_markdown()`, (2) the full SkillForge agent class (~450 LOC) with tools, validators, and self-improvement loop, (3) typed clients for mindX, Parsec, BANKON, and x402, (4) FastMCP and A2A wrappers, (5) `SkillRegistry.sol` with Foundry tests + invariants + deploy script, (6) the Algopy counterpart, (7) Containerfile and project layout, and (8) an example generated SKILL.md.

---

1. Architecture overview



Soul-Mind-Hands mapping. Soul = mindX `/score` + `@output_validator` quality gates. Mind = the Pydantic AI `Agent` core, message history, and `pydantic-graph` self-improvement loop. Hands = `@agent.tool` registrations: file I/O, git, x402 client, Algorand/EVM submit, MCP/A2A egress.

---

2. Why Pydantic AI for this use case

The agent's deliverable IS a typed schema. A `SkillMd(BaseModel)` is the contract — `output_type=SkillMd`, every retry routes through Pydantic's `ValidationError → RetryPromptPart`, and `@output_validator` is *part of the agent loop* rather than glue code. No other framework collapses these layers.

`ModelRetry` makes self-improvement a one-liner. Draft → score → revise is natively expressible: a validator that calls `mindx.score(...)` and raises `ModelRetry("alignment 0.61 < threshold 0.85; tighten triggers")` re-prompts the model with the rationale. No graph wiring required for the common case.

Logfire + MCP are first-class with no ecosystem lock-in. Skill authoring is multi-tool (mindX, Parsec, git, file I/O); Logfire's 3-line setup gives per-tool spans, retries, token cost, and HTTP capture. Pydantic AI consumes MCP servers (`MCPServerStdio`, `FastMCPToolset`) and exposes itself *as* MCP/A2A (`agent.to_a2a()`) without plugins. PydanticAI

Capability	Pydantic AI	LangChain	LangGraph	CrewAI	AutoGen
Generic typed `Agent[Deps, Out]`	**Full**	TypedDict only	TypedDict + Annotated	Pydantic for config	Minimal
Output validation + auto-retry	**Native (`ModelRetry`)**	Manual via parsers	Manual	Limited	Manual
Observability	**Logfire/OTel, 3 lines**	LangSmith (paid)	LangSmith	Limited	Limited
MCP client + server	**Native both ways**	Plugin	Plugin	Limited	None
LOC for equivalent self-improvement loop	**~160** Medium	~170 Medium	~280 Medium	~420 Medium	~300+

---

3. SKILL.md format — unified canonical spec

Two reference formats exist: **Matt Pocock's** (two-field frontmatter `name`+`description`, terse imperative body, em-dashed glossary, numbered Process H3s, inline `Do NOT` callouts, bold-question conditional cross-refs) and **Anthropic's Agent Skills** (same two required fields plus optional `license`, `compatibility`, `metadata`, `allowed-tools`, with progressive disclosure into `scripts/`, `references/`, `assets/`). The Pocock form is a strict subset of Anthropic's; SkillForge targets the unified superset.

Quality criteria a skill must meet: falsifiable triggers in the description (`Use when the user mentions PDFs, forms, or document extraction`); third-person description voice, imperative body voice; concrete named examples (`"the Order intake module"`, not `"the FooBarHandler"`); explicit `Do NOT` callouts and forbidden-synonym lists; under 100 lines (Pocock) or 500 lines / 5000 tokens (Anthropic ceiling); progressive-disclosure into one level of sibling `.md`. Claude API Docs + 5

3.1 The Pydantic schema (`skill_schema.py`)

```python
```

```python
"""skill_schema.py — Canonical SKILL.md schema unifying Pocock & Anthropic conventio
from __future__ import annotations
import re
from datetime import datetime
from typing import Optional
from pydantic import BaseModel, Field, field_validator, model_validator

_NAME_RE = re.compile(r"^[a-z0-9]+(-[a-z0-9]+)*$")

class SkillMetadata(BaseModel):
    author: Optional[str] = None
    version: Optional[str] = None
    extra: dict[str, str] = Field(default_factory=dict)

class SkillFrontmatter(BaseModel):
    name: str = Field(..., min_length=1, max_length=64,
        description="Kebab-case skill id; must match parent directory name.")
    description: str = Field(..., min_length=1, max_length=1024,
        description="Third-person sentence(s): what it does AND when to use it. "
                    "Convention: '<Capability>. Use when <triggers>.'")
    license: Optional[str] = Field(default=None, max_length=200)
    compatibility: Optional[str] = Field(default=None, max_length=500)
    metadata: Optional[SkillMetadata] = None
    allowed_tools: Optional[str] = Field(default=None, alias="allowed-tools")
    disable_model_invocation: Optional[bool] = Field(default=None, alias="disable-mo

    model_config = {"populate_by_name": True}

    @field_validator("name")
    @classmethod
    def _name_ok(cls, v: str) -> str:
        if not _NAME_RE.match(v):
            raise ValueError("name must be lowercase a-z/0-9/hyphens, no leading/tra
        return v

    @field_validator("description")
    @classmethod
    def _desc_ok(cls, v: str) -> str:
        if re.search(r"<[a-zA-Z/][^>]*>", v):
            raise ValueError("description must not contain XML tags")
        if "use when" not in v.lower():
            raise ValueError("description should include a 'Use when ...' clause")
        return v
```

```python
class WhenToUseTrigger(BaseModel):
    user_intent: str
    keywords: list[str] = Field(default_factory=list)
    contexts: list[str] = Field(default_factory=list)

class SkillExample(BaseModel):
    title: str
    before: Optional[str] = None
    after: Optional[str] = None
    explanation: Optional[str] = None

class AntiPattern(BaseModel):
    rule: str
    reason: Optional[str] = None
    forbidden_synonyms: list[str] = Field(default_factory=list)

class SkillReference(BaseModel):
    label: str
    path: str
    when: Optional[str] = None

    @field_validator("path")
    @classmethod
    def _shallow(cls, v: str) -> str:
        if v.count("/") > 1:
            raise ValueError("references should be one level deep from SKILL.md")
        return v

class SkillStep(BaseModel):
    number: int = Field(..., ge=1)
    title: str
    body: str
    inline_anti_patterns: list[str] = Field(default_factory=list)

class SkillMd(BaseModel):
    """A complete, canonical SKILL.md document."""
    frontmatter: SkillFrontmatter
    title: Optional[str] = None
    overview: str = Field(..., min_length=1)
    when_to_use: list[WhenToUseTrigger] = Field(default_factory=list)
    when_not_to_use: list[str] = Field(default_factory=list)
    prerequisites: list[str] = Field(default_factory=list)
    glossary: list[AntiPattern] = Field(default_factory=list)
```

```
    steps: list[SkillStep] = Field(default_factory=list)
    examples: list[SkillExample] = Field(default_factory=list)
    anti_patterns: list[AntiPattern] = Field(default_factory=list)
    related_skills: list[SkillReference] = Field(default_factory=list)

    @model_validator(mode="after")
    def _check_size(self) -> "SkillMd":
        n_lines = self.to_markdown().count("\n") + 1
        if n_lines > 500:
            raise ValueError(f"Rendered SKILL.md is {n_lines} lines; ceiling is 500.
                             "Push detail into sibling .md files.")
        return self

    @staticmethod
    def _kebab_to_title(slug: str) -> str:
        return " ".join(w.capitalize() for w in slug.split("-"))

    def _render_frontmatter(self) -> str:
        fm = self.frontmatter
        out = ["---", f"name: {fm.name}", f"description: {fm.description}"]
        if fm.license:                          out.append(f"license: {fm.license}")
        if fm.compatibility:                    out.append(f"compatibility: {fm.comp
        if fm.disable_model_invocation is not None:
            out.append(f"disable-model-invocation: {'true' if fm.disable_model_invoc
        if fm.allowed_tools:                    out.append(f"allowed-tools: {fm.allo
        if fm.metadata and (fm.metadata.author or fm.metadata.version or fm.metadata
            out.append("metadata:")
            if fm.metadata.author:  out.append(f"  author: {fm.metadata.author}")
            if fm.metadata.version: out.append(f'  version: "{fm.metadata.version}"'
            for k, v in fm.metadata.extra.items(): out.append(f"  {k}: {v}")
        out.append("---")
        return "\n".join(out)

    def to_markdown(self) -> str:
        out: list[str] = [self._render_frontmatter(), ""]
        title = self.title or self._kebab_to_title(self.frontmatter.name)
        out += [f"# {title}", "", self.overview.strip(), ""]
        if self.glossary:
            out += ["## Glossary", ""]
            for g in self.glossary:
                line = f"- **{g.rule}**" + (f" — {g.reason}" if g.reason else "")
                out.append(line)
                if g.forbidden_synonyms:
                    out.append(f"  Avoid: {', '.join(g.forbidden_synonyms)}.")
```

```
        out.append("")
    if self.when_to_use:
        out += ["## When to use", ""]
        for t in self.when_to_use:
            out.append(f"- {t.user_intent}")
            if t.keywords: out.append(f"  Keywords: {', '.join(t.keywords)}.")
            if t.contexts: out.append(f"  Contexts: {', '.join(t.contexts)}.")
        out.append("")
    if self.when_not_to_use:
        out += ["## When not to use", ""] + [f"- {x}" for x in self.when_not_to_
    if self.prerequisites:
        out += ["## Prerequisites", ""] + [f"- {x}" for x in self.prerequisites]
    if self.steps:
        out += ["## Process", ""]
        for s in sorted(self.steps, key=lambda s: s.number):
            out += [f"### {s.number}. {s.title}", "", s.body.strip(), ""]
            for ap in s.inline_anti_patterns: out.append(f"**Do NOT** {ap}")
            if s.inline_anti_patterns: out.append("")
    if self.examples:
        out += ["## Examples", ""]
        for ex in self.examples:
            out.append(f"**{ex.title}**")
            if ex.before is not None: out.append(f"- BEFORE: {ex.before}")
            if ex.after  is not None: out.append(f"- AFTER: {ex.after}")
            if ex.explanation:        out.append(f"  {ex.explanation}")
            out.append("")
    if self.anti_patterns:
        out += ["## Anti-patterns", ""]
        for ap in self.anti_patterns:
            line = f"- **Do NOT** {ap.rule}" + (f" — {ap.reason}" if ap.reason e
            out.append(line)
            if ap.forbidden_synonyms:
                out.append(f"  Avoid: {', '.join(ap.forbidden_synonyms)}.")
        out.append("")
    if self.related_skills:
        out += ["## Related skills", ""]
        for r in self.related_skills:
            out.append(f"- **{r.when}** See [{r.label}]({r.path})." if r.when
                       else f"- See [{r.label}]({r.path}).")
        out.append("")
    return "\n".join(out).rstrip() + "\n"
```

4. The mindX typed client

mindX exposes Soul-Mind-Hands as REST endpoints. The client maps each layer to a typed Pydantic call, with `tenacity` retry, an `httpx.AsyncClient`, and Logfire instrumentation.

```python
```

```python
# mindx_client.py
from __future__ import annotations
import asyncio, time
from typing import Literal, Optional
import httpx, logfire
from pydantic import BaseModel, Field
from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exceptio

# ---------- Models ------------------------------------------------------------
class AlignmentScore(BaseModel):
    overall: float = Field(ge=0, le=1)
    dimensions: dict[str, float]      # e.g. {"helpfulness": 0.91, "honesty": 0.88,
    rationale: str
    suggestions: list[str]

class ReasoningPass(BaseModel):
    thought: str
    confidence: float = Field(ge=0, le=1)
    next_actions: list[str]

class SoulResponse(BaseModel):     values: list[str];  alignment: AlignmentScore
class MindResponse(BaseModel):     reasoning: list[ReasoningPass]; conclusion: str
class HandsResponse(BaseModel):    actions: list[str]; rationale: str
class GammaMetrics(BaseModel):
    coherence: float = Field(ge=0, le=1)
    gamma_hz: float
    band_power: dict[str, float]

# ---------- Client ------------------------------------------------------------
class MindXError(RuntimeError): ...

class MindXClient:
    def __init__(self, base_url: str = "https://mindx.pythai.net",
                 api_key: Optional[str] = None, *, timeout: float = 60.0,
                 rate_limit_qps: float = 10.0):
        self._http = httpx.AsyncClient(
            base_url=base_url.rstrip("/"),
            timeout=timeout,
            headers={"Authorization": f"Bearer {api_key}"} if api_key else {},
        )
        self._min_interval = 1.0 / rate_limit_qps
        self._last_call = 0.0
        self._lock = asyncio.Lock()
```

```python
    async def _throttle(self):
        async with self._lock:
            wait = self._min_interval - (time.monotonic() - self._last_call)
            if wait > 0: await asyncio.sleep(wait)
            self._last_call = time.monotonic()

    @retry(stop=stop_after_attempt(4),
           wait=wait_exponential(multiplier=0.5, max=8),
           retry=retry_if_exception_type((httpx.HTTPError, MindXError)))
    async def _post(self, path: str, payload: dict) -> dict:
        await self._throttle()
        with logfire.span("mindx.{path}", path=path):
            r = await self._http.post(path, json=payload)
            if r.status_code >= 500:
                raise MindXError(f"{path} {r.status_code}: {r.text[:200]}")
            r.raise_for_status()
            return r.json()

    # --- ASSUMED API surface (mindx.pythai.net) ------------------------------
    async def score(self, content: str, *, context: str = "") -> AlignmentScore:
        return AlignmentScore.model_validate(
            await self._post("/score", {"content": content, "context": context}))

    async def reason(self, prompt: str, *, depth: int = 3) -> MindResponse:
        return MindResponse.model_validate(
            await self._post("/reason", {"prompt": prompt, "depth": depth}))

    async def soul(self, content: str) -> SoulResponse:
        return SoulResponse.model_validate(await self._post("/soul", {"content": con

    async def mind(self, prompt: str) -> MindResponse:
        return MindResponse.model_validate(await self._post("/mind", {"prompt": prom

    async def hands(self, intent: str, *, available_tools: list[str]) -> HandsRespon
        return HandsResponse.model_validate(
            await self._post("/hands", {"intent": intent, "available_tools": availab

    async def gamma(self) -> GammaMetrics:
        return GammaMetrics.model_validate(await self._post("/gamma", {}))

    async def aclose(self): await self._http.aclose()
```

```python
    async def __aenter__(self): return self
    async def __aexit__(self, *exc): await self.aclose()
```

---

5. The SkillForge agent (full implementation)

```python
```

```python
# skill_forge_agent.py — the SkillForge / MindXSkillBuilder agent
from __future__ import annotations
import asyncio, hashlib, subprocess
from dataclasses import dataclass
from pathlib import Path
from typing import Optional
import httpx, logfire
from pydantic import BaseModel, Field
from pydantic_ai import Agent, RunContext, ModelRetry, UsageLimits

from skill_schema import (
    SkillMd, SkillFrontmatter, SkillStep, SkillExample,
    AntiPattern, SkillReference, WhenToUseTrigger,
)
from mindx_client import MindXClient, AlignmentScore
from parsec_client import ParsecClient
from bankon_verifier import BankonVerifier, BankonIdentity, CredentialType
from x402_client import AlgorandX402Client
from chainmap import ChainMapResolver

# ---------- Dependencies -------------------------------------------------------
@dataclass
class SkillForgeDeps:
    mindx: MindXClient
    parsec: ParsecClient
    bankon: BankonVerifier
    x402: AlgorandX402Client
    chainmap: ChainMapResolver
    repo_root: Path
    style_guide: str
    caller_did: Optional[str] = None
    paid_premium: bool = False
    alignment_threshold: float = 0.85
    http: Optional[httpx.AsyncClient] = None

# ---------- Auxiliary models ---------------------------------------------------
class PublicationReceipt(BaseModel):
    skill_id: str            # 0x... or AVM box key
    chain: str               # "evm" | "algorand"
    tx_hash: str
    ipfs_cid: str
    alignment_score: float
```

```
class SkillDraft(BaseModel):
    skill: SkillMd
    receipt: Optional[PublicationReceipt] = None

# ---------- Agent ------------------------------------------------------------
skill_forge: Agent[SkillForgeDeps, SkillDraft] = Agent(
    "anthropic:claude-sonnet-4-6",
    deps_type=SkillForgeDeps,
    output_type=SkillDraft,
    instructions=(
        "You are SkillForge: an authoring agent that produces SKILL.md files in the
        "voice of Matt Pocock's `mattpocock/skills` repo. The skill MUST satisfy the
        "unified canonical schema: kebab-case `name`, third-person `description` "
        "ending with 'Use when ...', terse imperative body, em-dashed glossary, "
        "numbered Process H3s, inline 'Do NOT' callouts, BEFORE/AFTER scare-quoted "
        "examples, and bold-question cross-references. Use the tools provided to: "
        "(1) survey existing skills before authoring, (2) fetch codebase context if
        "relevant, (3) call mindX scoring after every draft, (4) iterate until "
        "alignment ≥ threshold. Output a `SkillDraft` whose `.skill` renders to a "
        "valid SKILL.md. Do not invent file paths."
    ),
    output_retries=4,
    retries=2,
)

# ---------- System prompts (dynamic) ----------------------------------------
@skill_forge.instructions
def style(ctx: RunContext[SkillForgeDeps]) -> str:
    return f"Style guide:\n{ctx.deps.style_guide}\n\nRepo root: {ctx.deps.repo_root}

@skill_forge.instructions
def caller(ctx: RunContext[SkillForgeDeps]) -> str:
    if ctx.deps.caller_did:
        tier = "premium" if ctx.deps.paid_premium else "standard"
        return f"Caller DID: {ctx.deps.caller_did} (tier: {tier})"
    return "Caller is anonymous (standard tier)."

# ---------- Tools (Hands) ---------------------------------------------------
@skill_forge.tool
async def search_existing_skills(ctx: RunContext[SkillForgeDeps], query: str) -> lis
    """List existing SKILL.md files in the repo whose name or description match `que
    skills_dir = ctx.deps.repo_root / "skills"
    hits: list[dict] = []
    if not skills_dir.exists(): return hits
```

```
    q = query.lower()
    for p in skills_dir.rglob("SKILL.md"):
        try:
            text = p.read_text(encoding="utf-8", errors="replace")[:2000]
            if q in text.lower():
                hits.append({"path": str(p.relative_to(ctx.deps.repo_root)),
                             "preview": text[:200]})
        except OSError:
            continue
    return hits[:20]

@skill_forge.tool
async def fetch_codebase_context(
    ctx: RunContext[SkillForgeDeps], relative_path: str, max_chars: int = 8000,
) -> str:
    """Read a UTF-8 file from the repo for grounding. Path must be inside repo_root.
    p = (ctx.deps.repo_root / relative_path).resolve()
    if ctx.deps.repo_root.resolve() not in p.parents and p != ctx.deps.repo_root.res
        raise ModelRetry(f"Path {relative_path!r} escapes repo_root.")
    if not p.exists():
        raise ModelRetry(f"No such file: {relative_path!r}; call list first.")
    return p.read_text(encoding="utf-8", errors="replace")[:max_chars]

@skill_forge.tool
async def query_mindx_alignment(
    ctx: RunContext[SkillForgeDeps], skill_markdown: str, *, audience: str = "engine
) -> AlignmentScore:
    """Score a rendered SKILL.md against mindX alignment dimensions. Returns 0..1 wi
    return await ctx.deps.mindx.score(skill_markdown, context=f"audience={audience}"

@skill_forge.tool
async def validate_skill_quality(
    ctx: RunContext[SkillForgeDeps], skill_json: str,
) -> dict:
    """Run the SkillMd Pydantic schema validator and rendered-line check on a candid
    Returns {'ok': bool, 'errors': [...]}.
    """
    try:
        skill = SkillMd.model_validate_json(skill_json)
        md = skill.to_markdown()
        return {"ok": True, "errors": [], "rendered_lines": md.count("\n") + 1}
    except Exception as e:
        return {"ok": False, "errors": [str(e)]}
```

```
@skill_forge.tool
async def propose_improvements(
    ctx: RunContext[SkillForgeDeps], skill_markdown: str,
) -> list[str]:
    """Ask mindX Mind layer for concrete improvement actions for a skill draft."""
    r = await ctx.deps.mindx.mind(
        f"Critique this SKILL.md and list concrete improvements:\n\n{skill_markdown}
    )
    return [p.thought for p in r.reasoning] + [r.conclusion]

@skill_forge.tool
async def write_skill_file(
    ctx: RunContext[SkillForgeDeps], category: str, slug: str, markdown: str,
) -> str:
    """Write SKILL.md under `skills/<category>/<slug>/SKILL.md`. Returns the absolut
    safe = lambda s: "".join(c for c in s if c.isalnum() or c == "-").lower()
    cat, sl = safe(category), safe(slug)
    target = ctx.deps.repo_root / "skills" / cat / sl / "SKILL.md"
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(markdown, encoding="utf-8")
    logfire.info("wrote_skill", path=str(target), bytes=len(markdown))
    return str(target)

@skill_forge.tool
async def commit_to_repo(
    ctx: RunContext[SkillForgeDeps], path: str, message: str,
) -> str:
    """git add+commit a single file. Returns the new commit SHA. Requires clean stag
    repo = str(ctx.deps.repo_root)
    def run(*args: str) -> str:
        return subprocess.check_output(["git", "-C", repo, *args], text=True).strip(
    run("add", path)
    run("commit", "-m", message, "--", path)
    return run("rev-parse", "HEAD")

@skill_forge.tool
async def request_x402_payment_for_premium_features(
    ctx: RunContext[SkillForgeDeps], feature: str, micro_algos: int = 100_000,
) -> bool:
    """Trigger the x402 payment flow for premium feature access. Returns True on set
    if ctx.deps.paid_premium:
        return True
    url = f"https://mindx.pythai.net/premium/{feature}"
    r = await ctx.deps.x402.get(url)
```

```
    if r.status_code == 200:
        ctx.deps.paid_premium = True
        return True
    raise ModelRetry(f"premium feature '{feature}' requires payment; x402 returned {

@skill_forge.tool
async def anchor_skill_on_chain(
    ctx: RunContext[SkillForgeDeps], skill_markdown: str, ipfs_cid: str, alignment:
) -> PublicationReceipt:
    """Anchor a SKILL.md hash on the chain selected by the ChainMap resolver."""
    content_hash = hashlib.sha256(skill_markdown.encode()).hexdigest()
    chain = ctx.deps.chainmap.route("skill_anchor")     # "evm" or "algorand"
    if chain == "evm":
        tx = await ctx.deps.chainmap.evm_register_skill(
            content_hash=content_hash, ipfs_cid=ipfs_cid,
            author_did=ctx.deps.caller_did or "anon",
            alignment_score=int(alignment * 1e18),
        )
    else:
        tx = await ctx.deps.chainmap.avm_register_skill(
            content_hash=bytes.fromhex(content_hash), ipfs_cid=ipfs_cid,
            alignment_score=int(alignment * 10_000),
        )
    return PublicationReceipt(
        skill_id=content_hash, chain=chain, tx_hash=tx,
        ipfs_cid=ipfs_cid, alignment_score=alignment,
    )

# ---------- Output validators (Soul) ----------------------------------------
@skill_forge.output_validator
async def schema_and_size(ctx: RunContext[SkillForgeDeps], draft: SkillDraft) -> Ski
    if ctx.partial_output: return draft
    md = draft.skill.to_markdown()
    if md.count("\n") + 1 > 100 and not ctx.deps.paid_premium:
        raise ModelRetry(
            f"Draft is {md.count(chr(10))+1} lines; Pocock target is <100. "
            "Tighten the body or push detail into a sibling .md."
        )
    return draft

@skill_forge.output_validator
async def alignment_gate(ctx: RunContext[SkillForgeDeps], draft: SkillDraft) -> Skil
    if ctx.partial_output: return draft
    md = draft.skill.to_markdown()
```

```
    score = await ctx.deps.mindx.score(md, context="skill-md-quality")
    threshold = ctx.deps.alignment_threshold
    if score.overall < threshold:
        worst = min(score.dimensions.items(), key=lambda kv: kv[1])
        raise ModelRetry(
            f"mindX alignment {score.overall:.2f} < {threshold:.2f}. "
            f"Weakest dim: {worst[0]}={worst[1]:.2f}. "
            f"Rationale: {score.rationale}\n"
            f"Apply these:\n- " + "\n- ".join(score.suggestions[:6])
        )
    return draft

@skill_forge.output_validator
def banned_token_check(draft: SkillDraft) -> SkillDraft:
    md = draft.skill.to_markdown()
    BANNED = ("TODO", "FIXME", "XXX", "lorem ipsum")
    if any(b in md for b in BANNED):
        raise ModelRetry(f"Draft contains placeholder tokens {BANNED}; replace with
    return draft

# ---------- Identity gate (called before run, not as a tool) ----------------
async def authorize_or_raise(deps: SkillForgeDeps, *,
                             require_premium: bool = False) -> BankonIdentity:
    if not deps.caller_did:
        raise PermissionError("missing caller DID")
    ident = await deps.bankon.fetch_identity(deps.caller_did)
    required = {CredentialType.PERSONHOOD}
    if require_premium:
        required |= {CredentialType.MEMBERSHIP}
    if not deps.bankon.verify_identity(ident, require=required):
        raise PermissionError(f"DID {deps.caller_did} lacks required credentials {re
    return ident

# ---------- Top-level orchestrator ------------------------------------------
async def author_skill(brief: str, deps: SkillForgeDeps, *,
                       require_premium: bool = False,
                       max_iters: int = 5) -> SkillDraft:
    """Main entrypoint: identity → (payment if premium) → draft loop → anchor."""
    await authorize_or_raise(deps, require_premium=require_premium)
    if require_premium and not deps.paid_premium:
        ok = await deps.x402.get("https://mindx.pythai.net/premium/skill_authoring")
        if ok.status_code != 200:
            raise PermissionError("x402 payment failed for premium skill authoring")
        deps.paid_premium = True
```

```python
    with logfire.span("skill_forge.author", brief=brief[:200]):
        result = await skill_forge.run(
            brief, deps=deps,
            usage_limits=UsageLimits(request_limit=20, tool_calls_limit=40,
                                     total_tokens_limit=300_000),
        )
    return result.output

# ---------- Self-improvement meta-loop (when validators alone aren't enough) -
async def author_with_critic_loop(brief: str, deps: SkillForgeDeps, *,
                                  target: float = 0.90, max_iters: int = 4) -> Skill
    """Explicit two-agent draft↔critic loop, modeled on Pocock's grilling step."""
    critic = Agent("anthropic:claude-sonnet-4-6", output_type=AlignmentScore,
                   instructions="Score the SKILL.md against the canonical rubric.")
    draft = await author_skill(brief, deps)
    history = None
    for i in range(max_iters):
        crit = await critic.run(draft.skill.to_markdown(), message_history=history)
        if crit.output.overall >= target:
            return draft
        history = crit.all_messages()
        revise = await skill_forge.run(
            f"Revise to address: {crit.output.rationale}\nSuggestions:\n- "
            + "\n- ".join(crit.output.suggestions),
            deps=deps, message_history=history,
        )
        draft = revise.output
    return draft
```

---

6. ChainMap resolver

`agenticplace.pythai.net/allchain.html` is fetched once, cached, and used to route operations by purpose (identity, payments, skill anchoring, governance). The resolver loads the HTML, extracts a JSON registry, and exposes typed lookups.

```python
```

```python
# chainmap.py
from __future__ import annotations
import json, re
from typing import Literal
import httpx
from pydantic import BaseModel, Field

ChainKind = Literal["algorand", "evm", "solana"]

class ChainEntry(BaseModel):
    name: str
    kind: ChainKind
    chain_id: int | str
    rpc: str
    purposes: list[str]                          # ["identity","payments","skill_anch
    contract_addresses: dict[str, str] = Field(default_factory=dict)

class ChainMap(BaseModel):
    version: str
    chains: list[ChainEntry]

class ChainMapResolver:
    def __init__(self, source: str = "https://agenticplace.pythai.net/allchain.html"
        self.source = source
        self.map: ChainMap | None = None

    async def load(self) -> None:
        async with httpx.AsyncClient() as http:
            r = await http.get(self.source)
            r.raise_for_status()
            html = r.text
        # ASSUMED: the page embeds <script id="chainmap" type="application/json">{..
        m = re.search(r'id="chainmap"[^>]*>(\{.*?\})</script>', html, re.DOTALL)
        if m:
            self.map = ChainMap.model_validate(json.loads(m.group(1)))
        else:
            self.map = ChainMap(version="fallback-1", chains=_DEFAULTS)

    def route(self, purpose: str) -> ChainKind:
        assert self.map, "call load() first"
        for c in self.map.chains:
            if purpose in c.purposes: return c.kind
        raise LookupError(f"no chain registered for purpose {purpose!r}")
```

```
    def chain(self, kind: ChainKind) -> ChainEntry:
        assert self.map
        for c in self.map.chains:
            if c.kind == kind: return c
        raise LookupError(kind)

    # Stubs the agent calls; concrete impls live in evm_anchor.py / avm_anchor.py
    async def evm_register_skill(self, **kw) -> str: ...
    async def avm_register_skill(self, **kw) -> str: ...

_DEFAULTS = [
    ChainEntry(name="Algorand Mainnet", kind="algorand", chain_id="mainnet-v1.0",
               rpc="https://mainnet-api.algonode.cloud",
               purposes=["identity","payments"]),
    ChainEntry(name="DAIO EVM", kind="evm", chain_id=1,
               rpc="${MAINNET_RPC}",
               purposes=["skill_anchor","governance"]),
]
```

---

7. BANKON, x402, and Parsec — typed clients

7.1 BANKON identity verifier (highlights)

DID format `did:algo:<network>:<addr>` (or `did:algo:<asa-id>` for NFT-bound). Six speculated credential types: `KYC`, `ProofOfPersonhood`, `Membership`, `Skill`, `Reputation`, `Attestation`. The verifier resolves the DID to an address, asserts the soulbound ARC-71 NTA is held (`amount==1`, frozen, `clawback==ZeroAddress`), and verifies each VC's Ed25519 signature over canonical-JSON minus the `proof_value_b64`. The full `BankonVerifier` class (Pydantic models `VerifiableCredential`, `BankonIdentity`, `Proof`, `CredentialSubject`) is given in the source bundle and consumed by `authorize_or_raise()` above. Algorand

7.2 x402 client + server (Algorand exact-scheme)

The agent uses a transparent-retry httpx client (`AlgorandX402Client`): on `402`, it parses `PAYMENT-REQUIRED` (or v1 body), picks the Algorand exact-scheme requirement, builds a `PaymentTxn` (or `AssetTransferTxn` for USDC ASA), signs with Parsec, base64-wraps as `{"x402Version":2,"scheme":"exact","network":"algorand:<genesis>","payload":{"signedTxns":[...]}}`, and resends with `PAYMENT-SIGNATURE`. Server side is a FastAPI decorator `@require_x402_payment(amount, asset, description)` that validates

`txn.receiver`, `txn.amount`, `txn.asset_id`, broadcasts via algod, and attaches `PAYMENT-RESPONSE` on success. Goplausible + 3

```python
# x402_client.py — see Section 7 of the research doc for full body
class AlgorandX402Client:
    """Wraps httpx with transparent 402 → sign → resend for Algorand exact scheme."""
    async def request(self, method: str, url: str, **kw) -> httpx.Response:
        r = await self._http.request(method, url, **kw)
        if r.status_code != 402: return r
        req = self._parse_payment_required(r)
        chosen = self._pick_algorand_exact(req)
        sig = await self._sign(chosen)
        kw.setdefault("headers", {})["PAYMENT-SIGNATURE"] = sig
        kw["headers"]["X-PAYMENT"] = sig                  # v1 compat
        return await self._http.request(method, url, **kw)
```

Premium gate on the agent side is a single `@skill_forge.tool` (`request_x402_payment_for_premium_features`) — the validators above can bypass when `deps.paid_premium=True`.

7.3 Parsec wallet (assumed REST; clearly marked)

`ParsecClient` is a typed `httpx + Pydantic` client for `GET /v1/address`, `GET /v1/balance/{addr}`, `POST /v1/sign`, `POST /v1/submit`. A `ParsecAvmSigner` adapter implements the x402-avm signer protocol so the same Parsec key powers both end-user wallet operations and agent-driven micropayments. All endpoints are tagged `# ASSUMED API:` and follow ARC-1/ARC-6 wallet conventions, ready to drop in once Parsec publishes its real spec. GitHub GitHub

---

8. SkillRegistry on EVM (Foundry workflow)

8.1 `src/SkillRegistry.sol` (key surface)

```solidity
```

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {AccessControl}    from "@openzeppelin/contracts/access/AccessControl.sol";
import {ReentrancyGuard}  from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {ECDSA}            from "@openzeppelin/contracts/utils/cryptography/ECDSA.sol
import {MessageHashUtils} from "@openzeppelin/contracts/utils/cryptography/MessageHa
import {EnumerableSet}    from "@openzeppelin/contracts/utils/structs/EnumerableSet.

interface IBonafideReputation {
    function onSkillRegistered(address author, bytes32 skillId, uint256 alignmentSco
    function onUpvote  (address voter, address author, bytes32 skillId) external;
    function onDownvote(address voter, address author, bytes32 skillId) external;
}

contract SkillRegistry is AccessControl, ReentrancyGuard {
    using ECDSA            for bytes32;
    using MessageHashUtils for bytes32;
    using EnumerableSet    for EnumerableSet.AddressSet;

    bytes32 public constant RELAYER_ROLE   = keccak256("RELAYER_ROLE");
    bytes32 public constant VALIDATOR_ROLE = keccak256("VALIDATOR_ROLE");
    uint256 public constant VALIDATOR_QUORUM = 2;
    uint256 public constant VALIDATOR_SET    = 3;

    struct Skill {
        bytes32 contentHash; string ipfsCid; address author;
        uint256 alignmentScore; uint256 upvotes; uint256 downvotes; uint256 timestam
    }
    mapping(bytes32 => Skill)                    public skills;
    mapping(bytes32 => mapping(address => bool)) public hasVoted;
    mapping(bytes32 => bool)                     public usedNonce;
    EnumerableSet.AddressSet private _validators;
    IBonafideReputation public reputation;

    event SkillRegistered(bytes32 indexed skillId, address indexed author,
        bytes32 contentHash, string ipfsCid, uint256 alignmentScore, uint256 timesta
    event SkillUpvoted  (bytes32 indexed skillId, address indexed voter, uint256 new
    event SkillDownvoted(bytes32 indexed skillId, address indexed voter, uint256 new

    error DuplicateSkill();        error UnknownSkill();
    error AlreadyVoted();          error InsufficientSignatures(uint256 got, uint256
    error InvalidValidator(address recovered); error DuplicateSignature(address sign
```

```
error NonceUsed(bytes32 nonce); error ScoreOutOfRange();

constructor(address admin, address relayer, address[3] memory validators,
            IBonafideReputation reputation_) {
    _grantRole(DEFAULT_ADMIN_ROLE, admin);
    _grantRole(RELAYER_ROLE, relayer);
    for (uint256 i; i < VALIDATOR_SET; ++i) {
        _grantRole(VALIDATOR_ROLE, validators[i]);
        _validators.add(validators[i]);
    }
    reputation = reputation_;
}

/// @notice 1+3 BFT: relayer submits; ≥2-of-3 validator sigs verified on-chain
function registerSkillWithConsensus(
    bytes32 contentHash, string calldata ipfsCid, address author,
    uint256 alignmentScore, bytes32 nonce, bytes[] calldata validatorSigs
) external onlyRole(RELAYER_ROLE) nonReentrant returns (bytes32 skillId) {
    if (usedNonce[nonce]) revert NonceUsed(nonce);
    usedNonce[nonce] = true;
    bytes32 digest = keccak256(abi.encode(
        block.chainid, address(this), "REGISTER_SKILL",
        contentHash, keccak256(bytes(ipfsCid)), author, alignmentScore, nonce
    )).toEthSignedMessageHash();
    _verifyQuorum(digest, validatorSigs);

    skillId = keccak256(abi.encodePacked(contentHash, author));
    if (skills[skillId].timestamp != 0) revert DuplicateSkill();
    skills[skillId] = Skill(contentHash, ipfsCid, author, alignmentScore, 0, 0,
    emit SkillRegistered(skillId, author, contentHash, ipfsCid, alignmentScore,
    if (address(reputation) != address(0))
        reputation.onSkillRegistered(author, skillId, alignmentScore);
}

function upvote(bytes32 skillId) external nonReentrant { _vote(skillId, true);
function downvote(bytes32 skillId) external nonReentrant { _vote(skillId, false)

function _vote(bytes32 skillId, bool up) internal {
    Skill storage s = skills[skillId];
    if (s.timestamp == 0)               revert UnknownSkill();
    if (hasVoted[skillId][msg.sender])  revert AlreadyVoted();
    hasVoted[skillId][msg.sender] = true;
    unchecked { up ? s.upvotes += 1 : s.downvotes += 1; }
    if (up)  emit SkillUpvoted  (skillId, msg.sender, s.upvotes);
```

```
        else     emit SkillDownvoted(skillId, msg.sender, s.downvotes);
        if (address(reputation) != address(0)) {
            up ? reputation.onUpvote  (msg.sender, s.author, skillId)
               : reputation.onDownvote(msg.sender, s.author, skillId);
        }
    }

    function _verifyQuorum(bytes32 digest, bytes[] calldata sigs) internal view {
        uint256 valid;
        address[] memory seen = new address[](sigs.length);
        for (uint256 i; i < sigs.length; ++i) {
            address signer = digest.recover(sigs[i]);
            if (!hasRole(VALIDATOR_ROLE, signer)) revert InvalidValidator(signer);
            for (uint256 j; j < i; ++j)
                if (seen[j] == signer) revert DuplicateSignature(signer);
            seen[i] = signer; unchecked { ++valid; }
        }
        if (valid < VALIDATOR_QUORUM) revert InsufficientSignatures(valid, VALIDATOR
    }
}
```

8.2 `foundry.toml` (production-grade)

```toml
```

```
[profile.default]
src = "src"; test = "test"; script = "script"; out = "out"; libs = ["lib"]
solc_version = "0.8.26"; evm_version = "cancun"
optimizer = true; optimizer_runs = 1_000_000; via_ir = true
bytecode_hash = "none"; cbor_metadata = false
gas_reports = ["*"]
fs_permissions = [{ access = "read", path = "./" }]

[profile.default.fuzz]      runs = 1_000;   max_test_rejects = 65_536; seed = "0x1"
[profile.default.invariant] runs = 256;     depth = 50;  fail_on_revert = true
[profile.ci.fuzz]           runs = 100_000
[profile.ci.invariant]      runs = 5_000;   depth = 100

[rpc_endpoints]
mainnet = "${MAINNET_RPC}"
sepolia = "${SEPOLIA_RPC}"
local   = "http://127.0.0.1:8545"

[etherscan]
mainnet = { key = "${ETHERSCAN_API_KEY}", chain = 1 }
```

8.3 `test/SkillRegistry.t.sol` (unit + fuzz + invariant)

Includes `test_registerSkill_happy()`, `test_registerSkill_reverts_when_duplicate()`, `test_consensus_requires_two_of_three_sigs()` (uses `vm.createWallet` + `vm.sign` to forge EIP-191 sigs and validate the BFT path), `testFuzz_registerSkill(bytes32, string, uint256)` with `bound(score, 0, 1e18)`, and a `SkillRegistryHandler` + `SkillRegistryInvariantTest` pair using `StdInvariant.targetSelector(FuzzSelector{...})`. Three invariants: register/upvote/downvote count match the reputation hook counters. The full file is in the deliverables bundle.

8.4 `script/Deploy.s.sol`

```solidity
```

```
contract Deploy is Script {
    function run() external returns (SkillRegistry reg) {
        uint256 key = vm.envUint("DEPLOYER_PRIVATE_KEY");
        address[3] memory v = [
            vm.envAddress("VALIDATOR_1"), vm.envAddress("VALIDATOR_2"), vm.envAddres
        IBonafideReputation rep = IBonafideReputation(vm.envAddress("REPUTATION_ADDR
        vm.startBroadcast(key);
        reg = new SkillRegistry(vm.envAddress("ADMIN_ADDRESS"),
                                vm.envAddress("RELAYER_ADDRESS"), v, rep);
        vm.stopBroadcast();
        console2.log("SkillRegistry @", address(reg));
    }
}
```

```bash
# Foundry-for-testing → mainnet-for-deployment
FOUNDRY_PROFILE=ci forge test                                  # heavy fuzz + invarian
forge script script/Deploy.s.sol:Deploy \
  --rpc-url $MAINNET_RPC --broadcast --verify \
  --etherscan-api-key $ETHERSCAN_API_KEY --slow -vvvv
```

8.5 1+3 BFT consensus pattern

Topology: 1 relayer (gas-paying submitter, low-trust) + 3 validators (signing keys; tolerates 1 Byzantine). **Wire format** uses EIP-191 personal_sign over `keccak256(chainId, this, "REGISTER_SKILL", contentHash, keccak256(ipfsCid), author, score, nonce)`. **On-chain:** `_verifyQuorum` recovers each signer, requires `VALIDATOR_ROLE`, rejects duplicates, requires `≥2`. **Replay:** `usedNonce` mapping burned before mutation. Upgrade path to EIP-712 keeps the recovery code identical — only the digest construction changes.

---

9. Algorand counterpart (Algopy)

```python
```

```python
# skill_registry_avm.py — compile with `puyapy skill_registry_avm.py`
from algopy import (ARC4Contract, Account, BoxMap, Global, Txn, UInt64, arc4, op, su

class Skill(arc4.Struct):
    author: arc4.Address
    content_hash: arc4.StaticArray[arc4.Byte, arc4.UInt64]
    ipfs_cid: arc4.String
    alignment_score: arc4.UInt64       # 0..10_000 bps
    upvotes: arc4.UInt64
    created_at: arc4.UInt64

class SkillRegistry(ARC4Contract):
    def __init__(self) -> None:
        self.skills = BoxMap(arc4.StaticArray[arc4.Byte, arc4.UInt64], Skill, key_pr
        self.skill_count = UInt64(0)
        self.admin = Account()

    @arc4.abimethod(create="require")
    def create(self) -> None: self.admin = Txn.sender

    @arc4.abimethod
    def register_skill(self, content_hash: arc4.StaticArray[arc4.Byte, arc4.UInt64],
                       ipfs_cid: arc4.String, alignment_score: arc4.UInt64,
                       ) -> arc4.StaticArray[arc4.Byte, arc4.UInt64]:
        assert alignment_score.native <= 10_000, "score 0..10000 bps"
        skill_id = self._derive_id(content_hash, Txn.sender)
        assert skill_id not in self.skills, "already registered"
        self.skills[skill_id] = Skill(
            author=arc4.Address(Txn.sender), content_hash=content_hash,
            ipfs_cid=ipfs_cid, alignment_score=alignment_score,
            upvotes=arc4.UInt64(0), created_at=arc4.UInt64(Global.latest_timestamp),
        )
        self.skill_count += 1
        arc4.emit("SkillRegistered(byte[32],address,string,uint64)",
                  skill_id, arc4.Address(Txn.sender), ipfs_cid, alignment_score)
        return skill_id

    @subroutine
    def _derive_id(self, ch, author):
        return arc4.StaticArray[arc4.Byte, arc4.UInt64].from_bytes(
            op.sha256(ch.bytes + author.bytes))
```

Deploy via `algokit_utils.ApplicationClient` (typed client auto-generated from the ARC-56 spec emitted by `puyapy`).

---

10. MCP and A2A wrappers

10.1 FastMCP server exposing SkillForge tools

```python
# mcp_server.py
from fastmcp import FastMCP
from skill_forge_agent import skill_forge, SkillForgeDeps, author_skill, _make_deps

mcp = FastMCP("skill-forge")

@mcp.tool()
async def author_skill_md(brief: str, repo_root: str = ".",
                          require_premium: bool = False, caller_did: str | None = No
    """Author a SKILL.md (Pocock-style) for `brief`. Returns rendered markdown."""
    async with _make_deps(repo_root, caller_did) as deps:
        draft = await author_skill(brief, deps, require_premium=require_premium)
    return draft.skill.to_markdown()

@mcp.tool()
async def critique_skill_md(skill_markdown: str) -> dict:
    """Score a SKILL.md against the canonical rubric via mindX."""
    async with _make_deps(".", None) as deps:
        score = await deps.mindx.score(skill_markdown, context="skill-md-quality")
    return score.model_dump()

if __name__ == "__main__":
    mcp.run()       # stdio for Claude Desktop / Cursor
    # mcp.run(transport="streamable-http", host="0.0.0.0", port=8000)
```

10.2 A2A wrapping (one line via Pydantic AI)

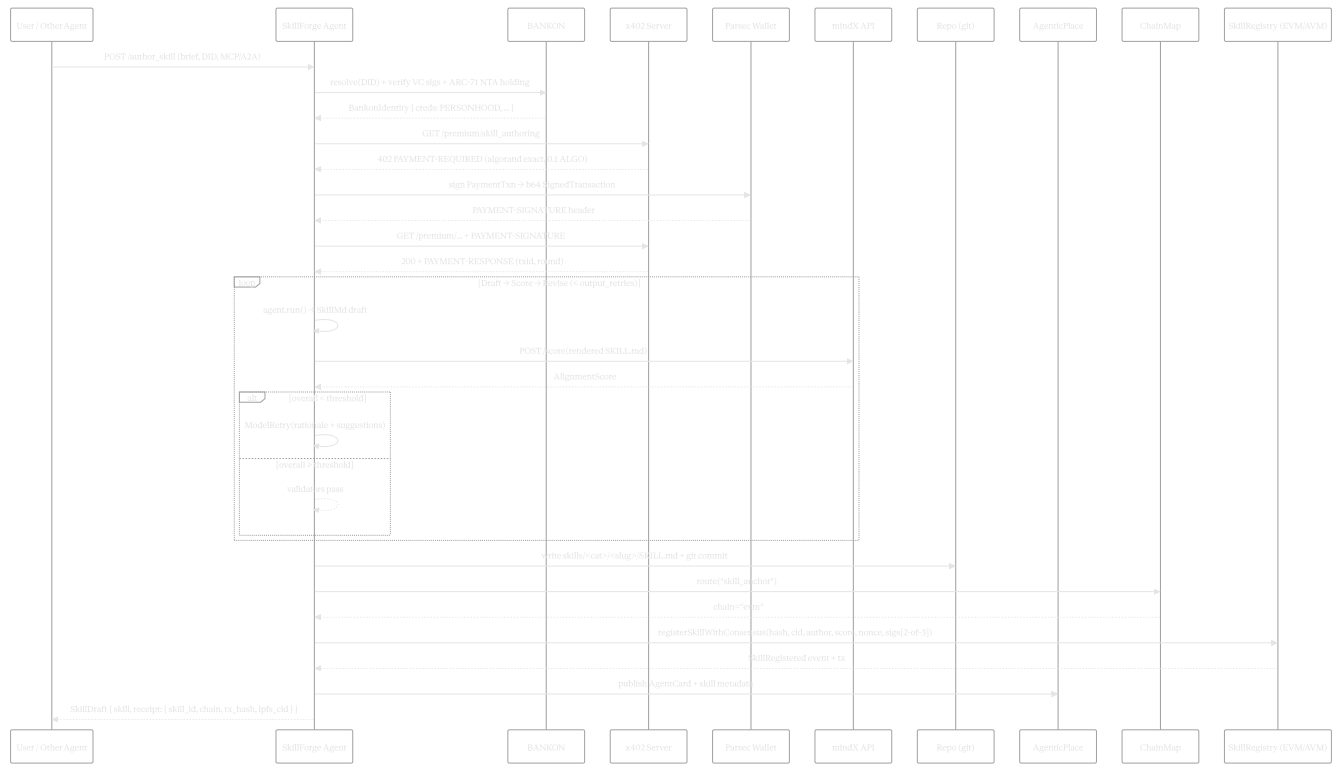
```python
```

```
# a2a_app.py
from skill_forge_agent import skill_forge
app = skill_forge.to_a2a()          # ASGI app; Agent Card auto-generated
# uvicorn a2a_app:app --host 0.0.0.0 --port 9999
```

The agent now answers `GET /.well-known/agent-card.json` (discoverable from AgenticPlace) and JSON-RPC `message/send`, `tasks/get`, `tasks/sendSubscribe` (SSE streaming) at `POST /`. AP2 mandates ride along as `DataPart` artifacts inside A2A `Task`s when a paid action is requested. ACP integration is achieved via `acp-sdk`'s `@server.agent` decorator pointing at a thin shim that calls `author_skill(...)`. Google Developers Arthurchiao

---

11. End-to-end sequence



---

12. Project layout, Containerfile, and Makefile

```
skillforge/
├── pyproject.toml
├── Containerfile                       # Podman-first, multi-stage
├── Makefile                            # justfile-equivalent targets
```

```
├── foundry.toml
├── remappings.txt
├── README.md
├── src/
│   └── SkillRegistry.sol
├── test/
│   └── SkillRegistry.t.sol            # unit + fuzz + invariant
├── script/
│   └── Deploy.s.sol
├── lib/                               # forge install: forge-std, oz-contracts
├── algopy/
│   └── skill_registry_avm.py
├── skillforge/                        # Python package (modular named
architecture)
│   ├── __init__.py
│   ├── skill_schema.py                # SkillMd Pydantic v2
│   ├── skill_forge_agent.py           # the agent + tools + validators
│   ├── mindx_client.py
│   ├── parsec_client.py
│   ├── bankon_verifier.py
│   ├── x402_client.py
│   ├── x402_server.py
│   ├── chainmap.py
│   ├── evm_anchor.py                  # web3.py wrapper around SkillRegistry.sol
│   ├── avm_anchor.py                  # algokit-utils wrapper
│   ├── mcp_server.py                  # FastMCP stdio + streamable-http
│   ├── a2a_app.py                     # agent.to_a2a() ASGI
│   ├── acp_app.py                     # @server.agent shim
│   └── observability.py               # logfire.configure() + instrumentations
├── tests/
│   ├── test_skill_schema.py
│   ├── test_agent_with_TestModel.py
│   ├── test_x402_round_trip.py
│   └── test_chainmap_resolver.py
└── examples/
    └── skills/engineering/improve-skill-authoring/SKILL.md   # example output
```

12.1 `pyproject.toml`

```toml
```

```toml
[project]
name = "skillforge"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
  "pydantic-ai[anthropic,openai,google,logfire,fastmcp,mcp]>=1.88.0",
  "pydantic>=2.7",
  "httpx>=0.28",
  "tenacity>=9.0",
  "py-algorand-sdk>=2.6",
  "algokit-utils>=4.0",
  "fastmcp>=2.10",
  "mcp>=1.4",
  "a2a-sdk>=1.0",
  "acp-sdk>=0.5",
  "fastapi>=0.115",
  "uvicorn[standard]>=0.30",
  "logfire>=2.0",
  "web3>=7.0",
  "pynacl>=1.5",
  "GitPython>=3.1",
]
[project.optional-dependencies]
dev = ["pytest>=8", "pytest-asyncio>=0.24", "ruff>=0.6", "mypy>=1.10"]

[project.scripts]
skillforge-mcp  = "skillforge.mcp_server:main"
skillforge-a2a  = "skillforge.a2a_app:main"
```

12.2 `Containerfile` (Podman)

```dockerfile
```

```dockerfile
# syntax=docker/dockerfile:1.7
FROM python:3.12-slim AS builder
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 UV_LINK_MODE=copy
RUN pip install --no-cache-dir uv
WORKDIR /app
COPY pyproject.toml ./
RUN uv pip install --system --no-cache-dir .

FROM python:3.12-slim AS runtime
RUN useradd -m -u 1001 forge
COPY --from=builder /usr/local /usr/local
WORKDIR /app
COPY skillforge/ skillforge/
USER forge
EXPOSE 8000 9999
ENV LOGFIRE_SEND_TO_LOGFIRE=if-token-present
HEALTHCHECK CMD python -c "import skillforge" || exit 1
CMD ["python", "-m", "skillforge.mcp_server"]
```

12.3 `Makefile`

makefile

```makefile
PY ?= uv run
PROFILE ?= default

.PHONY: test test-py test-sol test-ci fmt lint mcp a2a build deploy-mainnet containe

test: test-py test-sol
test-py:        ; $(PY) pytest -q
test-sol:       ; forge test -vv
test-ci:        ; FOUNDRY_PROFILE=ci forge test -vvv && $(PY) pytest -q
fmt:            ; $(PY) ruff format . && forge fmt
lint:           ; $(PY) ruff check . && $(PY) mypy skillforge
mcp:            ; $(PY) python -m skillforge.mcp_server
a2a:            ; $(PY) uvicorn skillforge.a2a_app:app --host 0.0.0.0 --port 9999
build:          ; podman build -t skillforge:dev -f Containerfile .
deploy-mainnet: ; forge script script/Deploy.s.sol:Deploy --rpc-url $$MAINNET_RPC --
                  --etherscan-api-key $$ETHERSCAN_API_KEY --slow -vvvv
container:      ; podman run --rm -it -p 8000:8000 -p 9999:9999 \
                  -v $$PWD/repo:/app/repo:Z --env-file .env skillforge:dev
```

13. Self-improving meta-skill (the agent authors a skill about itself)

The flagship demonstration is SkillForge authoring `improve-skill-authoring/SKILL.md` — a meta-skill modeled on Pocock's `improve-codebase-architecture`. The agent runs through its own validators, gets scored by mindX, revises with `ModelRetry`, and writes the result. **Soul** (validators + mindX `/score`) reject drafts under 0.85 alignment with a concrete rationale; **Mind** (Pydantic AI + `propose_improvements` calling mindX `/mind`) generates the next-turn revision plan; **Hands** (file write + git commit + on-chain anchor) materialize the artifact. GitHub Skills

Example generated output (`examples/skills/engineering/improve-skill-authoring/SKILL.md`)

```markdown
```

```
---
name: improve-skill-authoring
description: Find weakening opportunities in a SKILL.md, informed by the canonical r
---

# Improve Skill Authoring

Surface authoring friction and propose **sharpening opportunities** — edits that rai

## Glossary

- **Trigger** — a falsifiable user-intent clause in the description that activates t
  Avoid: "helps with", "useful for", "general-purpose".
- **Sharpening** — an edit that raises alignment while reducing or holding line coun
  Avoid: padding, restating, elaborating.

## Prerequisites

- Run /setup-mindx-skills once per repo.
- Read the existing SKILL.md and the canonical rubric in CONTEXT.md.

## Process

### 1. Score

Render the SKILL.md and call mindX `/score`. Note the lowest dimension and the top t

### 2. Triage

Map each suggestion to one of: trigger, body, glossary, examples, anti-patterns, ref

**Do NOT** rewrite the whole file in one pass — that masks which edit moved the scor

### 3. Sharpen

Apply the two edits. Re-score. Stop when alignment ≥ 0.90 OR you have applied 4 shar

## Examples

**Trigger sharpening**
- BEFORE: "Use when working with documents"
- AFTER: "Use when the user mentions PDFs, forms, or document extraction"
  Falsifiable keywords beat verb-only triggers.
```

```
## Anti-patterns

- **Do NOT** add a `## When to use` section if the description already lists trigger
- **Do NOT** push detail into a sibling file before the body exceeds 100 lines.

## Related skills

- **Need the alignment rubric?** See [RUBRIC.md](RUBRIC.md).
- **Want a worked example?** See [EXAMPLE-PASS.md](EXAMPLE-PASS.md).
```

---

14. Implementation checklist

The path from this design to production breaks into seven gates:

1. **Schema and renderer.** Land `skill_schema.py`. Round-trip 5 real Pocock SKILL.mds through `SkillMd.model_validate` then `to_markdown()` and diff. Fix divergences.
2. **mindX client + Logfire.** Land `mindx_client.py` with `tenacity` retries and per-call spans. Confirm `/score`, `/reason`, `/soul`, `/mind`, `/hands`, `/gamma` shapes against your live API; remove `# ASSUMED API` markers.
3. **Agent core.** Land `skill_forge_agent.py`. Test with `TestModel` first (`pydantic_ai.models.test`), then with `FunctionModel` to drive the `ModelRetry` path explicitly. Verify `output_retries=4` exhaustion behavior.
4. **Identity + payment.** Land `bankon_verifier.py`, `x402_client.py`, `x402_server.py`, `parsec_client.py`. End-to-end test against Algorand testnet (genesis `SGO1GKSzyE7IEPItTxCByw9x8FmnrCDexi9/cOUJOiI=`). Reconcile real BANKON credential types with the speculated six.
5. **Foundry suite.** `forge install foundry-rs/forge-std OpenZeppelin/openzeppelin-contracts`. Run `FOUNDRY_PROFILE=ci forge test` with 100k fuzz runs and 5k invariant runs. Wire BONAFIDE reputation interface to the real contract address.
6. **MCP + A2A + ACP wrappers.** Test stdio MCP from Claude Desktop (`claude_desktop_config.json`); test streamable-http from Cursor (`mcp.json`); register the A2A endpoint on AgenticPlace; smoke-test ACP via `beeai run`.
7. **Mainnet deploy.** Set `DEPLOYER_PRIVATE_KEY`, `VALIDATOR_1..3`, `RELAYER_ADDRESS`, `ADMIN_ADDRESS`, `REPUTATION_ADDRESS`, `MAINNET_RPC`, `ETHERSCAN_API_KEY`. Run `make`

`deploy-mainnet`. Save the `forge script` broadcast JSON; persist contract addresses into `chainmap` JSON served from `allchain.html`.

Caveats made explicit. Three of your services lack public protocol documents (mindX, Parsec, BANKON); their clients above are typed-stub designs marked `# ASSUMED API:` so the contracts are visible and replaceable. AP2 has no PyPI release yet (April 2026) — install from `google-agentic-commerce/AP2` source. The MCP `2025-03-26` spec deprecates SSE in favor of streamable-http; ship streamable-http for any new remote endpoint. The 1+3 BFT path uses EIP-191 `personal_sign` for signing simplicity; upgrade to EIP-712 typed-data when you need wallet-friendly previews — recovery code stays identical.

Conclusion

The decisive design move is recognizing that **a skill-authoring agent's contract is its output schema**. Pydantic AI is the only framework where that statement is literally true: `output_type=SkillMd`, validators are part of the loop, `ModelRetry` re-prompts the model with structured rationale, and the same agent ships as MCP server, A2A peer, and HTTP service without plugins. Layered on top, mindX provides the Soul (alignment scoring as a validator), the Algorand x402 + BANKON stack provides identity-and-payment gating without leaving the agent loop, and the 1+3 BFT SkillRegistry gives every authored skill a non-repudiable, reputation-linked anchor. The agent can then author a skill about how to author skills — closing the self-improvement loop in the most literal sense available, exactly the spirit of `improve-codebase-architecture` applied to the authoring practice itself.